

INTELIGENCIA ARTIFICIAL Y MINI ROBOTS

REDES NEURONALES ARTIFICIALES

Desarrollo de los ejercicios propuestos en la sección 5.9

Ejercicio 1. Modelo de un experimento con una red de dos capas ocultas

Ejercicio 2. Fashion MNIST con una red de dos capas ocultas

Ejercicio 3. Fashion MNIST: funciones e instrucciones explicadas

Ejercicio 4. Modelo con un dataset de Kaggle y ejemplos de internet

Ejercicio 5 (punto libre). De la convolución a la red convolucional

Presentado por:

Edinson Sanchez Fuentes (edsanchezf@unal.edu.co)

Brayan Alejandro Muñoz Pérez (bmunozp@unal.edu.co)

Joan Sebastián Roberto Puerto (jroberto@unal.edu.co)

Docente: José J. Martínez P.

Universidad Nacional de Colombia, Sede Bogotá

Septiembre de 2026

Contenido	
1. Introducción.....	4
1.1 Metodología.....	4
2. Ejercicio 1. Modelo de un experimento con una red de dos capas ocultas	5
2.1 El experimento.....	5
2.2 Diseño de la red	6
2.3 Estructura del programa.....	7
2.4 Selección de las épocas y entrenamiento	7
2.5 Comparación con modelos de referencia	8
2.6 Predicción con datos nuevos.....	9
2.7 Verificación con TensorFlow.....	10
2.8 Conclusiones.....	10
3. Ejercicio 2. Fashion MNIST con una red de dos capas ocultas	11
3.1 Datos.....	11
3.2 La red de dos capas ocultas.....	11
3.3 Prueba del modelo: la precisión obtenida	12
3.4 ¿Qué aporta la segunda capa oculta?	13
3.5 Análisis de errores y de las capas ocultas	14
3.6 Conclusiones.....	15
4. Ejercicio 3. Fashion MNIST: funciones e instrucciones explicadas	16
4.1 Planteamiento.....	16
4.2 Funciones e instrucciones principales.....	16
4.3 Resultados.....	18
4.4 Conclusiones.....	19
5. Ejercicio 4. Modelo con un dataset de Kaggle y ejemplos de internet	20
5.1 Elección del conjunto de datos.....	20
5.2 Características y rótulo	20
5.3 Calidad de los datos	21
5.4 Modelo: red con dos capas ocultas	21
5.5 Ejemplos tomados de internet.....	22
5.6 Análisis de los resultados.....	23
5.7 Conclusiones.....	23
6. Ejercicio 5 (punto libre). De la convolución a la red convolucional.....	25
6.1 Planteamiento.....	25
6.2 Convolución y pooling desde cero.....	25
6.3 Red convolucional	25
6.4 Dentro de la red entrenada	27
6.5 Conclusiones.....	27
7. Uso de herramientas de IA Generativa	28

7.1 Herramientas empleadas	28
7.2 Prompts utilizados.....	28
7.3 Alcance y limitaciones observadas	28
8. Programas, repositorio y ejecución.....	30
8.1 Estructura de los archivos	30
8.2 Ejecución en Google COLAB.....	30
8.3 Ejecución local con Jupyter	30
9. Bibliografía.....	31

1. Introducción

Este documento presenta el desarrollo de los cinco ejercicios propuestos en la sección 5.9 del capítulo *Introducción a las Redes Neuronales Artificiales*. Cada ejercicio se entrega como un notebook de Jupyter ejecutado, con sus resultados incluidos, y en este documento se describen sus partes, sus resultados y las conclusiones que se derivan de ellos, conforme a los parámetros de entrega del curso.

1.1 Metodología

Una red neuronal siempre produce una salida, y sin un punto de comparación no es posible distinguir un modelo que funciona de uno que solo produce números razonables. Las verificaciones realizadas fueron las siguientes:

- **Comprobación de las derivadas.** El gradiente calculado por la retropropagación programada se comparó con derivadas numéricas por diferencias centrales.
- **Repetición de las operaciones de TensorFlow.** Con los pesos extraídos de las redes entrenadas se repitió la propagación con NumPy, para confirmar que la librería calcula exactamente las ecuaciones del capítulo.
- **Líneas base.** Cada red se comparó con un modelo más simple o con un modelo de referencia: una neurona sin capas ocultas, una red con una sola capa oculta, un modelo físico o una regla sencilla.
- **Robustez.** Cuando el resultado podía depender de la inicialización, el entrenamiento se repitió con varias semillas aleatorias.
- **Reproducción del material del capítulo.** Los ejemplos numéricos de la sección 5.8 (convolución, max pooling y figura 5.24) se reprodujeron antes de construir sobre ellos.

2. Ejercicio 1. Modelo de un experimento con una red de dos capas ocultas

Enunciado: tome una tabla de datos de un experimento cualquiera. Encuentre un modelo con base en una red neuronal de dos capas ocultas. Dele datos a ese modelo para hacer una predicción.

2.1 El experimento

Se tomó la tabla del experimento de cinética enzimática con puromicina de Treloar (1974), un conjunto clásico de laboratorio publicado por Bates y Watts (1988) y distribuido con el lenguaje R como Puromycin. Se midió la velocidad inicial de una reacción catalizada por una enzima, en cuentas/min², para seis concentraciones del sustrato, en ppm, con dos grupos de células: tratadas con el antibiótico puromicina y sin tratar. Cada condición se midió dos veces, salvo la concentración mayor en las células sin tratar, de modo que la tabla tiene 23 filas.

Tabla 1. Tabla de datos del experimento (velocidad en cuentas/min²).

Concentración (ppm)	Células tratadas	Células sin tratar
0.02	76, 47	67, 51
0.06	97, 107	84, 86
0.11	123, 139	98, 115
0.22	159, 152	131, 124
0.56	191, 201	144, 158
1.10	207, 200	160

El experimento se escogió por tres razones: es una tabla real de laboratorio, pequeña y completa, que cabe en el propio notebook; la relación entre las variables no es lineal, porque la velocidad crece rápido y luego se satura; y existe un modelo físico conocido para esta reacción, la ecuación de Michaelis-Menten, que sirve como referencia independiente para juzgar lo que aprenda la red. El problema es de predicción (regresión): la salida es un número.

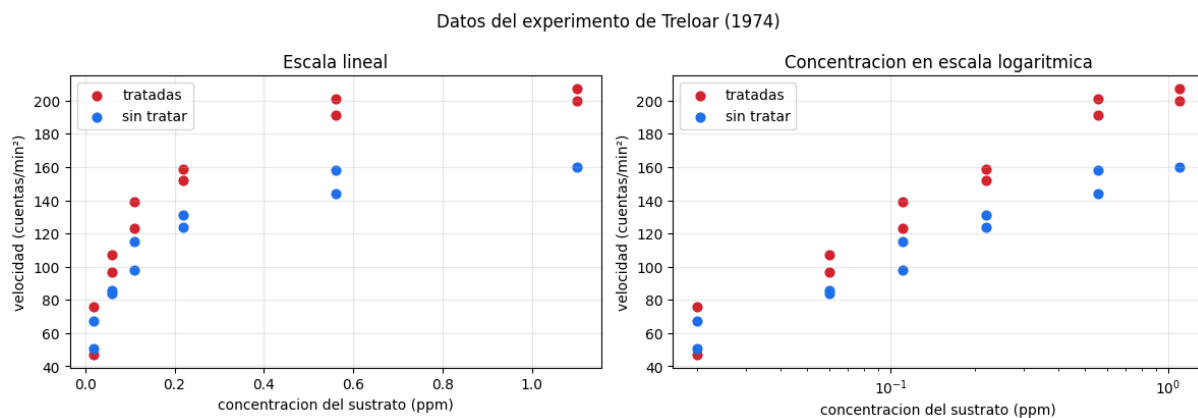


Figura 1. Datos del experimento. A la derecha, la concentración en escala logarítmica.

La figura muestra la saturación típica de una reacción enzimática y que las células tratadas alcanzan velocidades mayores. También muestra el ruido experimental: las dos repeticiones de una misma condición pueden diferir bastante (76 y 47 en la primera fila), y ningún modelo puede predecir una medición nueva con un error menor que esa variabilidad.

2.2 Diseño de la red

Características. Se usaron dos, $X = [x_1, x_2]^T$: el logaritmo de la concentración, estandarizado, y el estado de las células (1 si están tratadas, 0 si no). Se usa el logaritmo porque las concentraciones abarcan un factor de 55 y están espaciadas aproximadamente en progresión geométrica; en esa escala los puntos quedan repartidos de forma pareja y la curva es más suave. La estandarización deja la entrada en la zona en que la sigmoide no está saturada. El rótulo, la velocidad, también se estandarizó, y las predicciones se devuelven a cuentas/min² con la transformación inversa.

Topología. La red tiene la estructura de la figura 5.13 del capítulo: entrada, dos capas ocultas y salida.

Tabla 2. Capas de la red 2-4-4-1.

Capa	Neuronas	Activación	Cálculo	Parámetros
0, entrada	2	—	$A^{(0)} = X$	—
1, primera capa oculta	4	sigmoide	$Z^{(1)} = W^{(1)}A^{(0)} + B^{(1)}, A^{(1)} = \sigma(Z^{(1)})$	$2 \cdot 4 + 4 = 12$
2, segunda capa oculta	4	sigmoide	$Z^{(2)} = W^{(2)}A^{(1)} + B^{(2)}, A^{(2)} = \sigma(Z^{(2)})$	$4 \cdot 4 + 4 = 20$
3, salida	1	lineal	$Z^{(3)} = W^{(3)}A^{(2)} + B^{(3)}, H(X) = Z^{(3)}$	$4 + 1 = 5$
Total				37

La única diferencia con las redes del capítulo está en la neurona de salida, que es lineal. La sigmoide solo produce valores entre 0 y 1, apropiados para una clase, pero aquí la salida es una velocidad. Las dos capas ocultas conservan la sigmoide, que es la que introduce la no linealidad.

Retropropagación. Se usaron las ecuaciones de la sección 5.5 del capítulo para una red de tres capas, con la función de pérdida $J(W) = (1/2m) \sum (h(X) - Y)^2$ que promedia el error cuadrático sobre los $m = 23$ ejemplos. Como la salida es lineal, su derivada vale 1 y el factor $\text{diag}(\sigma'(Z^{(3)}))$ se reemplaza por la identidad:

$$\begin{aligned}
 e &= Yr - A^{(3)}, \quad \delta^{(3)} = e / m \\
 \delta^{(2)} &= \text{diag}(\sigma'(Z^{(2)})) W^{(3)T} \delta^{(3)}, \quad \delta^{(1)} = \text{diag}(\sigma'(Z^{(1)})) W^{(2)T} \delta^{(2)} \\
 \partial J / \partial W^{(k)} &= -\delta^{(k)} A^{(k-1)T}, \quad \partial J / \partial B^{(k)} = -\delta^{(k)}, \quad W^{(k)} \leftarrow W^{(k)} - \eta \partial J / \partial W^{(k)}
 \end{aligned}$$

La derivada de la sigmoide es $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, la expresión de la sección 5.2.2. Los 23 ejemplos se procesan a la vez: X es una matriz de 2×23 cuyas columnas son los ejemplos, y el producto $\delta^{(k)} A^{(k-1)T}$ suma las contribuciones de todos ellos. La red tiene más parámetros que datos (37 frente a 23), así que puede memorizar el ruido; por eso el número de épocas se escogió con validación cruzada y la red se comparó con modelos más simples.

2.3 Estructura del programa

Tabla 3. Secciones del notebook del Ejercicio 1.

Sección	Contenido
1. El experimento	Descripción, variables y justificación de la elección.
2. Tabla de datos	Los 23 datos en un DataFrame de pandas y su gráfica.
3. Diseño de la red	Preparación de las características; funciones sig, dSig, inicializa, propaga, perdida, backpropagation y entrena, con los nombres del programa del Ejemplo 1 extendidos a dos capas ocultas.
4. Verificación del gradiente	Comparación de las 37 derivadas con diferencias centrales.
5. Selección de épocas	Validación cruzada dejando uno fuera, con tres semillas.
6. Entrenamiento final	Curva de pérdida, ajuste, residuos y pesos aprendidos.
7. Referencias	Michaelis-Menten, neurona sin capas ocultas, error puro; validación cruzada de todos los modelos; robustez con 10 semillas.
8. Predicción	Predicciones para concentraciones no medidas, efecto del tratamiento y extrapolación.
9. Verificación con TensorFlow	La misma red en Keras, con los mismos pesos iniciales.
10. Conclusiones	

El gradiente calculado por backpropagation coincidió con el numérico en los 37 parámetros, con un error relativo máximo de 1.0×10^{-8} .

2.4 Selección de las épocas y entrenamiento

Con 23 datos no conviene apartar un conjunto de validación. Se usó validación cruzada dejando uno fuera: la red se entrena 23 veces, cada vez sin una de las mediciones, y se mide el error al predecir la que quedó por fuera, como raíz del error cuadrático medio (RMSE). Se registró este error en varios puntos del entrenamiento, con tres inicializaciones y $\eta = 0.5$.

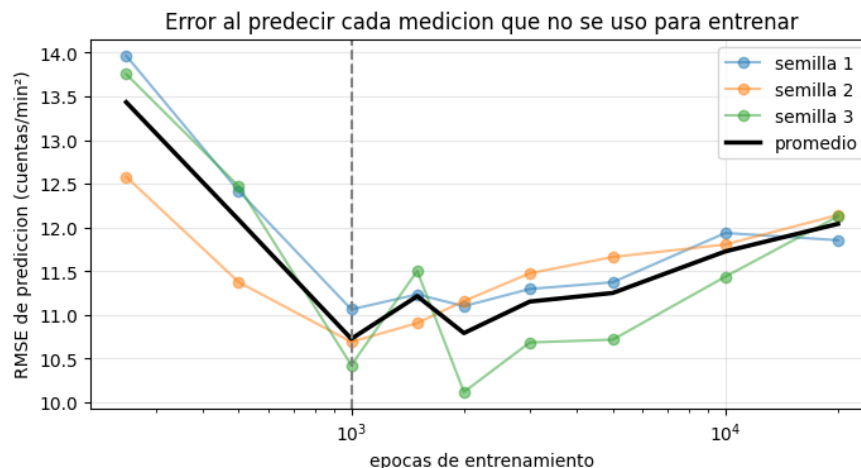


Figura 2. Error de predicción en validación cruzada según el número de épocas.

La curva tiene la forma de la figura 5.12 del capítulo. Con 250 épocas el error promedio es de 13.4 cuentas/min² (subentrenamiento); baja a 10.7 en 1 000 épocas y vuelve a subir hasta 12.0 en 20 000, cuando la red empieza a ajustar el ruido (sobreentrenamiento). Se eligieron 1 000 épocas; entre 1 000 y 5 000 el error se mantiene entre 10.7 y 11.3, así que la elección no es crítica. Con ellas, la pérdida baja de 0.64 a 0.013 y el error sobre los propios datos de entrenamiento es de 7.5 cuentas/min².

2.5 Comparación con modelos de referencia

Que la red se ajuste a los datos de entrenamiento no dice si es un buen modelo. Se comparó, con la misma validación cruzada, con el modelo físico de Michaelis-Menten, $v = V_{\max} \cdot c / (K + c)$, ajustado por mínimos cuadrados no lineales para cada grupo de células; con una neurona sin capas ocultas y salida lineal, que es una regresión lineal con las mismas dos características; y con el error puro del experimento, la desviación estándar entre repeticiones, que es el límite de lo que cualquier modelo puede predecir.

Tabla 4. Error de predicción sobre datos no vistos (validación cruzada dejando uno fuera).

Modelo	Parámetros	RMSE (cuentas/min ²)
Promedio de todas las mediciones	1	48.58
Neurona lineal sin capas ocultas	3	12.64
Michaelis-Menten (modelo físico)	4	11.26
Red 2-4-4-1 con dos capas ocultas	37	11.06
Error puro entre repeticiones (límite inferior)		9.97

Los parámetros ajustados de Michaelis-Menten, $V_{\max} = 212.7$ y $K = 0.0641$ para las células tratadas, y $V_{\max} = 160.3$ y $K = 0.0477$ para las no tratadas, coinciden con los publicados para este conjunto. La tabla lleva a tres conclusiones: las capas ocultas son necesarias, porque la neurona sin ellas tiene un error de 12.6; la red predice tan bien como el modelo físico (11.1 frente a 11.3) sin que se le haya dado la forma de la ecuación; y ambas están cerca del límite del experimento, 10.0. La diferencia de 0.2 entre la red y el modelo físico no es significativa con 23 datos, y el valor de la red es levemente optimista porque la misma validación cruzada se usó para escoger el número de épocas.

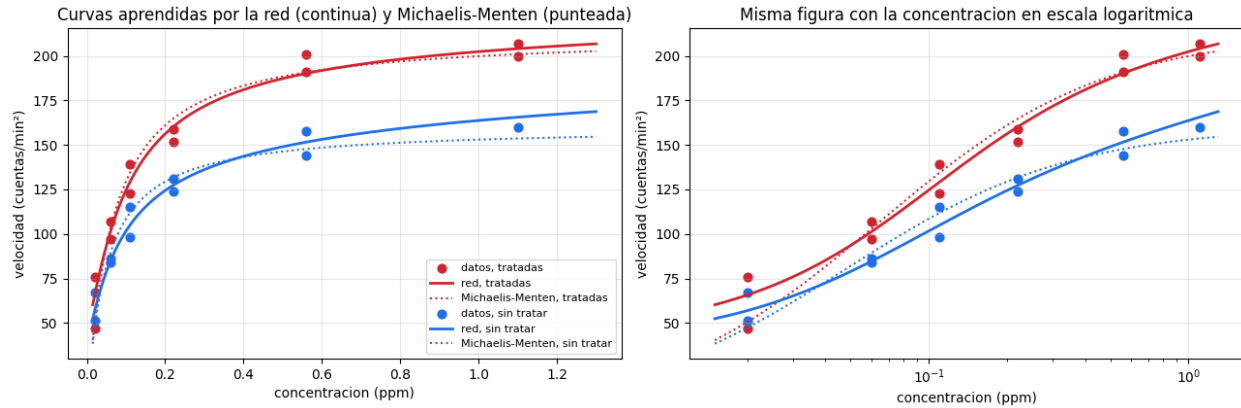


Figura 3. Curvas aprendidas por la red (continuas) y curvas de Michaelis-Menten (punteadas).

Dentro del rango medido las curvas de la red y del modelo físico prácticamente se superponen. Se separan en los extremos: en las células sin tratar a concentración alta, donde solo tres mediciones fijan la curva, y en la concentración más baja, donde las repeticiones están muy dispersas. Con diez inicializaciones distintas, el error de ajuste va de 7.3 a 8.4 y la predicción a 0.30 ppm para células tratadas queda entre 168.6 y 173.8, con una desviación de 1.6: el resultado no depende de la semilla.

2.6 Predicción con datos nuevos

Para cumplir la última parte del enunciado se le dieron a la red siete concentraciones que no se midieron en el experimento, para los dos grupos de células.

Tabla 6. Predicciones de la red para concentraciones nuevas (cuentas/min²).

Concentración (ppm)	Tratadas: red	Tratadas: Michaelis-Menten	Sin tratar: red	Sin tratar: Michaelis-Menten
0.04	85.2	81.7	72.5	73.1
0.08	114.1	118.1	94.1	100.4
0.15	143.7	149.0	115.3	121.6
0.30	171.7	175.2	136.1	138.3
0.40	181.0	183.3	143.6	143.2
0.80	198.1	196.9	159.3	151.3
1.00	202.4	199.9	163.7	153.0

En 12 de los 14 casos la red y el modelo físico difieren en menos de 7 cuentas/min², por debajo del error puro de las mediciones. La red permite además responder una pregunta experimental: el tratamiento con puromicina aumenta la velocidad en 13 cuentas/min² (18 %) a 0.04 ppm y en unas 39 cuentas/min² a partir de 0.8 ppm, un aumento relativo que se estabiliza entre 24 y 26 % desde 0.15 ppm.

Fuera del rango medido la situación cambia. Según el modelo físico la velocidad no puede superar V_{max} (213 y 160), pero la red sigue subiendo: a 20 ppm predice 231 para las tratadas y

209 para las no tratadas, y tiende a unos 242 para ambos grupos, con lo que el efecto del tratamiento desaparece. La red no conoce la ley física; solo aprendió una curva que pasa cerca de los puntos entre 0.02 y 1.10 ppm. Interpola bien, pero no se debe usar para extrapolar.

2.7 Verificación con TensorFlow

Se construyó la misma red 2-4-4-1 en Keras, se le cargaron los mismos pesos iniciales y se entrenó con descenso del gradiente simple usando los 23 ejemplos en cada paso. Como Keras usa la pérdida mse, que es el doble de J, se usó una tasa de aprendizaje de $\eta/2$. Después de 1 000 épocas los pesos de las dos versiones difieren en menos de 3.3×10^{-6} y las predicciones en una diez milésima: la red programada con las ecuaciones del capítulo y la red de Keras recorren exactamente el mismo camino de entrenamiento.

2.8 Conclusiones

1. **Se construyó un modelo de un experimento real con una red de dos capas ocultas.** La red 2-4-4-1 predice la velocidad de la reacción a partir de la concentración y del tratamiento con un error de 11.1 cuentas/min² sobre datos no vistos.
2. **La red aprendió la ley del experimento sin conocerla.** Su error es igual al del modelo de Michaelis-Menten y ambos están cerca del error puro de las mediciones.
3. **Las capas ocultas son las que representan la no linealidad:** sin ellas el error sube a 12.6.
4. **Con pocos datos, el número de épocas es decisivo.** La red tiene más parámetros que datos; la validación cruzada mostró el subentrenamiento y el sobreentrenamiento de la figura 5.12 y permitió detener el entrenamiento a tiempo.
5. **La predicción es confiable dentro del rango de los datos y no fuera de él.** Entre 0.04 y 1.00 ppm la red y el modelo físico coinciden y estiman que el tratamiento aumenta la velocidad hasta en un 26 %; por encima de 1.10 ppm la red viola la saturación de la reacción.
6. **La implementación es correcta:** coincide con el gradiente numérico y con el entrenamiento de la misma red en TensorFlow.

3. Ejercicio 2. Fashion MNIST con una red de dos capas ocultas

Enunciado: con base en la librería tensorflow, descargue el data set fashion MNIST. Haga una clasificación de prendas de vestir. Use una red neuronal con dos capas ocultas y pruebe el modelo, ¿qué precisión obtuvo? Saque conclusiones.

3.1 Datos

Fashion MNIST conserva el formato del MNIST de dígitos de la sección 5.4 del capítulo: imágenes en escala de grises de $28 \times 28 = 784$ píxeles, 60 000 de entrenamiento, 10 000 de prueba y 10 clases perfectamente balanceadas, pero cada imagen es una prenda o un accesorio. Se descarga con `tf.keras.datasets.fashion_mnist.load_data()`. Los píxeles se dividieron por 255 y se usaron tres conjuntos, como indica el capítulo: 50 000 imágenes para entrenar, 10 000 para validación (decidir cuándo parar) y 10 000 de prueba, que se usaron una sola vez al final.



Figura 4. Dos ejemplos de cada una de las 10 clases.

3.2 La red de dos capas ocultas

```
tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(name="aplanar"),
    tf.keras.layers.Dense(256, activation="relu", name="oculta_1"),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(128, activation="relu", name="oculta_2"),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation="softmax", name="salida")])
```

Tabla 7. Capas de la red 784-256-128-10.

Capa	Qué calcula	Parámetros
Flatten	imagen $28 \times 28 \rightarrow$ vector X de 784	0
oculta_1: Dense(256, ReLU)	$A^{(1)} = \text{ReLU}(W^{(1)T}X + B^{(1)})$	$784 \cdot 256 + 256 = 200\,960$
oculta_2: Dense(128, ReLU)	$A^{(2)} = \text{ReLU}(W^{(2)T}A^{(1)} + B^{(2)})$	$256 \cdot 128 + 128 = 32\,896$
salida: Dense(10, softmax)	$A^{(3)} = \text{softmax}(W^{(3)T}A^{(2)} + B^{(3)})$	$128 \cdot 10 + 10 = 1\,290$
Total		235 146

Las dos capas Dropout apagan al azar el 20 % de las neuronas de cada capa oculta durante el entrenamiento para reducir el sobreentrenamiento; no tienen pesos y no actúan al predecir, así que no son capas ocultas. El notebook cuenta las capas con pesos del modelo y confirma que hay

exactamente dos ocultas, *oculta_1* y *oculta_2*, más la de salida. Se usó el optimizador Adam ($\eta = 0.001$), la entropía cruzada $E = -\ln p(y)$ como pérdida, lotes de 32 imágenes y parada temprana: el entrenamiento se detiene cuando la pérdida de validación no mejora en 5 épocas y se restauran los pesos de la mejor.

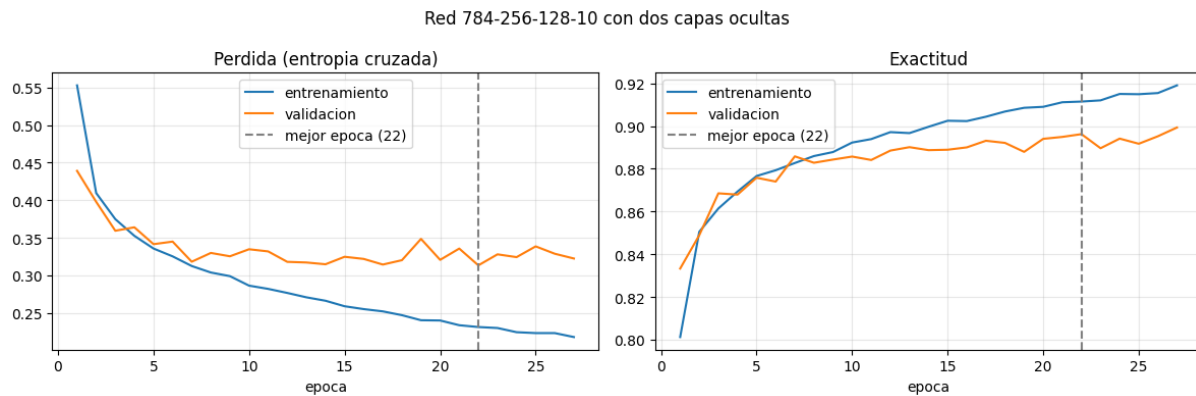


Figura 5. Pérdida y exactitud durante el entrenamiento de la red de dos capas ocultas.

La mejor época fue la 22, con 89.6 % de exactitud en validación y 91.1 % en entrenamiento; el entrenamiento se detuvo en la 27. La distancia creciente entre las curvas es el sobreentrenamiento de la figura 5.17 del capítulo, que el *Dropout* retrasa y la parada temprana corta a tiempo.

3.3 Prueba del modelo: la precisión obtenida

La red de dos capas ocultas clasificó correctamente 8 872 de las 10 000 prendas de prueba: una exactitud de 88.7 %, con un intervalo de confianza del 95 % de 88.1 % a 89.3 %. Un clasificador al azar acertaría el 10 %.

En el lenguaje común *precisión* se usa para la proporción de aciertos, que en clasificación se llama exactitud (*accuracy*); en aprendizaje de máquina la precisión (*precision*) mide, de las imágenes asignadas a una clase, qué fracción era de esa clase. Se reportan ambas: la precisión promedio por clase también fue de 88.7 %, igual que la sensibilidad y el F1, porque las clases están balanceadas.

Tabla 7. Desempeño por clase de la red de dos capas ocultas en el conjunto de prueba.

Clase	Precisión	Sensibilidad	F1
Pantalón	0.991	0.974	0.982
Bolso	0.983	0.971	0.977
Sandalia	0.985	0.957	0.971
Botín	0.954	0.968	0.961
Tenis	0.949	0.964	0.956
Vestido	0.872	0.896	0.884
Camiseta/top	0.804	0.868	0.835
Abrigo	0.811	0.798	0.804
Buzo	0.787	0.815	0.801

Clase	Precisión	Sensibilidad	F1
Camisa	0.734	0.661	0.696
Promedio	0.887	0.887	0.887

Verificación. Con las tres matrices de pesos extraídas con `get_weights()` se repitió la propagación con NumPy: $A^{(1)} = \text{ReLU}(XW^{(1)} + B^{(1)})$, $A^{(2)} = \text{ReLU}(A^{(1)}W^{(2)} + B^{(2)})$, $A^{(3)} = \text{softmax}(A^{(2)}W^{(3)} + B^{(3)})$. Las probabilidades coinciden con las de Keras con una diferencia máxima de 1.3×10^{-6} y las 10 000 predicciones son idénticas.

3.4 ¿Qué aporta la segunda capa oculta?

Se entrenaron en las mismas condiciones tres redes, cada una con tres semillas: sin capas ocultas (784-10), con una capa oculta de 300 neuronas (784-300-10) y la red de dos capas ocultas. La de una capa se diseñó con casi el mismo número de parámetros que la de dos (238 510 frente a 235 146), para medir el efecto de la profundidad y no el de tener más pesos.

Tabla 8. Exactitud en prueba según el número de capas ocultas (tres semillas: 2026, 7 y 42).

Arquitectura	Parámetros	Exactitud por semilla	Promedio	Desviación
0 capas ocultas (784-10)	7 850	84.17 / 84.27 / 84.40 %	84.28 %	0.12
1 capa oculta (784-300-10)	238 510	88.20 / 87.87 / 88.05 %	88.04 %	0.17
2 capas ocultas (784-256-128-10)	235 146	88.72 / 87.68 / 87.90 %	88.10 %	0.55

La primera capa oculta aporta casi 4 puntos de exactitud (de 84.3 % a 88.0 %) y baja la pérdida de prueba de 0.45 a 0.35: es la diferencia entre un modelo lineal y uno no lineal. La segunda capa, con los mismos parámetros, no cambia el resultado de forma medible: 88.10 % frente a 88.04 %, una diferencia de 0.06 puntos, mucho menor que la variación entre semillas de la propia red de dos capas (de 87.7 % a 88.7 %). La red principal, con la semilla 2026, obtuvo el mejor resultado de las nueve corridas; el valor que mejor representa a la arquitectura es el promedio, 88.1 %.

3.5 Análisis de errores y de las capas ocultas

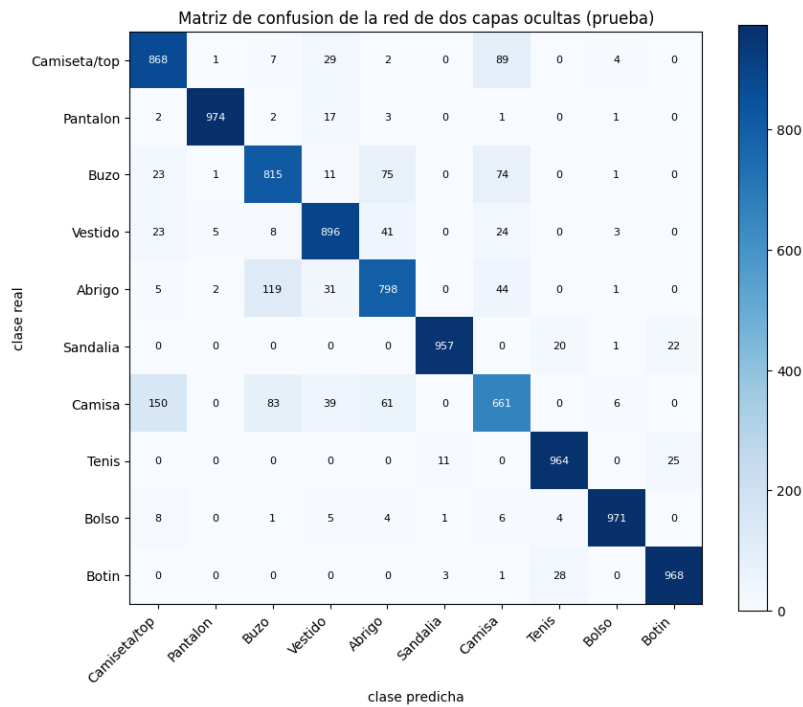


Figura 6. Matriz de confusión de la red de dos capas ocultas en el conjunto de prueba.

De los 1 128 errores, 732 (65 %) ocurren entre las cuatro prendas superiores: camiseta, buzo, abrigo y camisa. Las confusiones más frecuentes son camisa $\rightarrow$ camiseta (150) y abrigo $\rightarrow$ buzo (119). La confianza de la red es informativa: es de 0.94 en promedio en los aciertos y de 0.69 en los errores, y cuando la probabilidad de la clase predicha supera 0.99, lo que ocurre en el 58 % de las imágenes, la red acierta el 99.0 % de las veces.

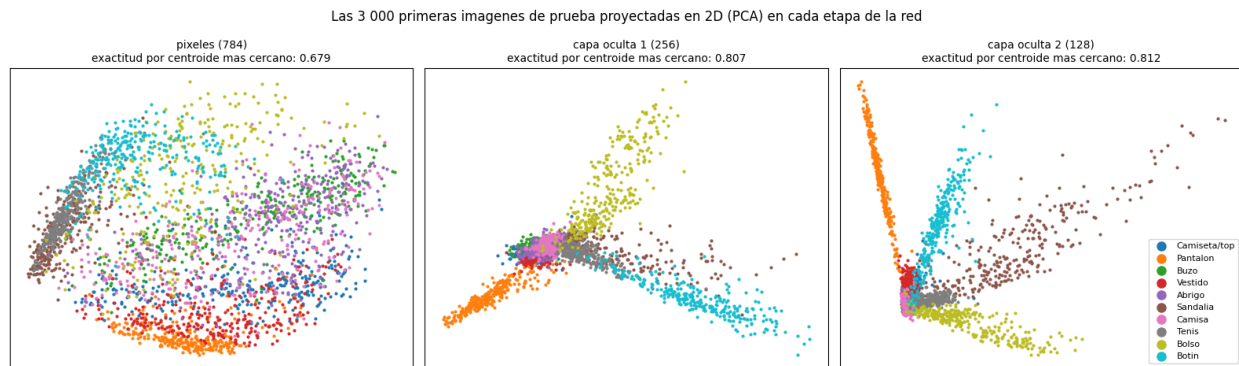


Figura 7. Imágenes de prueba proyectadas en dos dimensiones (PCA) en los píxeles y a la salida de cada capa oculta.

Para ver qué hacen las capas ocultas se midió qué tan agrupadas quedan las clases en cada etapa, con un clasificador que asigna cada imagen a la clase de centroide más cercano. Sobre los píxeles acierta el 67.9 %; sobre la salida de la primera capa oculta, el 80.7 %; sobre la de la segunda, el 81.2 %. Las capas ocultas transforman los datos en una representación en la que las clases están

más separadas; la mayor parte de la transformación la hace la primera capa, lo que concuerda con la comparación anterior.

3.6 Conclusiones

1. **Precisión obtenida: 88.7 %.** La red 784-256-128-10 clasificó correctamente 8 872 de las 10 000 prendas de prueba, con una precisión promedio por clase también de 88.7 %. Con otras semillas la exactitud va de 87.7 % a 88.7 %, con un promedio de 88.1 %.
2. **La red tiene exactamente dos capas ocultas** y calcula las ecuaciones de propagación del capítulo: la propagación a mano reproduce las 10 000 predicciones de TensorFlow.
3. **Las capas ocultas son imprescindibles, pero la segunda aporta poco en este problema.** De cero a una capa oculta la exactitud sube 3.8 puntos; de una a dos, con los mismos parámetros, 0.06, dentro de la variación entre semillas. Más profundidad no es automáticamente mejor.
4. **Las capas ocultas reorganizan los datos** de modo que las clases quedan más separadas: el clasificador por centroides pasa de 67.9 % sobre los píxeles a 81.2 % sobre la segunda capa oculta.
5. **El error se concentra en prendas parecidas**, y la camisa es la clase más difícil (66 % de sensibilidad). Una red densa trata cada píxel como una característica independiente y no aprovecha la estructura espacial de la imagen; el Ejercicio 5 aborda esa limitación con una red convolucional.
6. **Buenas prácticas.** El conjunto de prueba se usó una sola vez, el número de épocas lo decidió la validación y las arquitecturas se compararon con varias semillas y con un número de parámetros similar.

4. Ejercicio 3. Fashion MNIST: funciones e instrucciones explicadas

Enunciado: con base en la librería tensorflow, descargue el data set fashion MNIST. Haga una clasificación de prendas de vestir. Explique cada una de las funciones y las principales instrucciones, saque conclusiones.

4.1 Planteamiento

Este ejercicio se centra en explicar el programa. Se usó el modelo del ejemplo con el que termina la sección 5.8 del capítulo, una red con una capa oculta de 128 neuronas, y se recorrió el programa completo, desde la descarga hasta la predicción. Después de cada celda de código el notebook explica sus instrucciones y, para las funciones principales, repite su cálculo con NumPy y comprueba que coincide con el de TensorFlow.

```
model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(10, activation="softmax")])
```

El capítulo escribe las activaciones como `tf.nn.relu` y `tf.nn.softmax`; aquí se escriben por su nombre, que produce el mismo cálculo y además permite guardar y volver a cargar el modelo.

4.2 Funciones e instrucciones principales

Tabla 10. Funciones utilizadas, qué hacen y cómo se comprobaron.

Función o instrucción	Qué hace	Comprobación en el notebook
<code>keras.datasets.fashion_mnist.load_data()</code>	Descarga los datos la primera vez y devuelve dos tuplas, (imágenes, rótulos) de entrenamiento y de prueba, como arreglos de NumPy uint8.	Formas (60000, 28, 28) y (10000, 28, 28); 6 000 imágenes por clase.
<code>imagenes / 255.0</code>	Normaliza los píxeles a [0, 1] para que $Z = W^T X + B$ no sea enorme y el descenso del gradiente sea estable.	El píxel 228 pasa a 0.894.
<code>keras.Sequential(...)</code>	Modelo de capas en secuencia: la red feedforward de la figura 5.13.	—
<code>keras.Input(shape=(28, 28))</code>	Declara la forma de cada ejemplo de entrada.	—
<code>layers.Flatten()</code>	Convierte la matriz de 28×28 en el vector X de 784 valores; no tiene pesos.	Igual a <code>reshape(1, 784)</code> .
<code>layers.Dense(n, activation=g)</code>	Capa completamente conectada: $A = g(W^T A_{\text{prev}} + B)$, la neurona artificial de la sección 5.2.2 aplicada a n neuronas.	Salida calculada a mano con <code>get_weights()</code> : diferencia de 9×10^{-8} .
<code>"relu"</code> , <code>"softmax"</code>	ReLU: $\max(0, z)$. Softmax: $e^{z_j} / \sum e^{z_i}$, diez probabilidades que suman 1.	Las diez salidas suman 1.
<code>model.summary()</code>	Capas, formas y número de parámetros.	$784 \cdot 128 + 128 + 128 \cdot 10 + 10 = 101\,770$.

Función o instrucción	Qué hace	Comprobación en el notebook
<code>model.compile(optimizer, loss, metrics)</code>	Define cómo se entrena: optimizador Adam, pérdida de entropía cruzada $-\ln p(y)$ y métrica de exactitud, que solo se reporta.	Pérdida inicial calculada a mano igual a la de Keras (2.51).
<code>tf.GradientTape, tape.gradient</code>	Graba las operaciones y aplica la regla de la cadena hacia atrás: la retropropagación automática.	Un gradiente por cada W y B, de la misma forma.
<code>SGD(...).apply_gradients</code>	Aplica $W \leftarrow W - \eta \partial E / \partial W$.	Pesos nuevos iguales a W $-\eta$ gradiente; la pérdida del lote baja de 2.59 a 2.00.
<code>model.fit(x, y, epochs, batch_size, validation_data, callbacks)</code>	Ciclo de entrenamiento: épocas de 1 563 lotes de 32 imágenes; evalúa la validación al final de cada época y devuelve el historial.	12 épocas; mejor época la 8.
<code>callbacks.EarlyStopping</code>	Detiene el entrenamiento si la pérdida de validación no mejora en 4 épocas y restaura los mejores pesos.	Detuvo en la época 12 y restauró la 8.
<code>keras.callbacks.Callback, on_epoch_end</code>	Permite ejecutar código propio en momentos del entrenamiento; aquí imprime un resumen por época.	—
<code>history.history</code>	Diccionario con la pérdida y las métricas por época, para graficarlas.	Claves loss, accuracy, val_loss, val_accuracy.
<code>model.evaluate(x, y)</code>	Pérdida y exactitud en el conjunto de prueba, sin tocar los pesos.	87.4 % de exactitud.
<code>model.predict(x), np.argmax</code>	Las diez probabilidades de cada imagen y la clase más probable.	Cálculo a mano igual a predict (2×10^{-8}).
<code>np.expand_dims(img, 0)</code>	Convierte una imagen en un lote de una imagen, porque Keras siempre procesa lotes.	$(28, 28) \rightarrow (1, 28, 28)$.
<code>tf.math.confusion_matrix</code>	Tabla de clase real contra clase predicha.	—
<code>model.save, keras.models.load_model</code>	Guarda y recupera el modelo entrenado.	Predicciones idénticas tras cargar.

4.3 Resultados

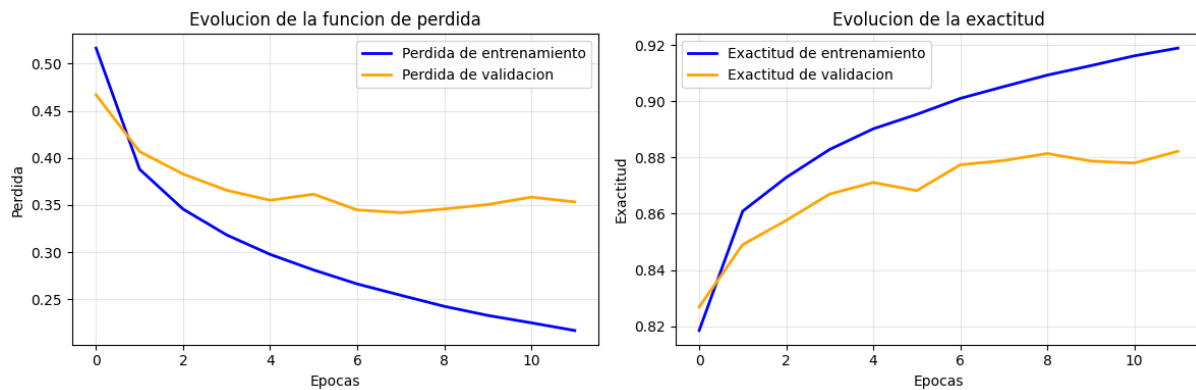


Figura 8. Pérdida y exactitud durante el entrenamiento del modelo del capítulo.

En la primera época la exactitud de validación ya es de 82.7 %. La mejor época fue la 8 (pérdida de validación de 0.342 y exactitud de 87.9 %); después la pérdida de validación sube mientras la de entrenamiento sigue bajando, y EarlyStopping detuvo el entrenamiento en la época 12 y restauró los pesos de la 8. En el conjunto de prueba la red clasificó correctamente el 87.4 % de las prendas (8 738 de 10 000), cerca de lo que anticipaba la validación. Por clase, los aciertos van del 97.1 % de los pantalones al 62.5 % de las camisas, seguidas de los abrigos (71.2 %).

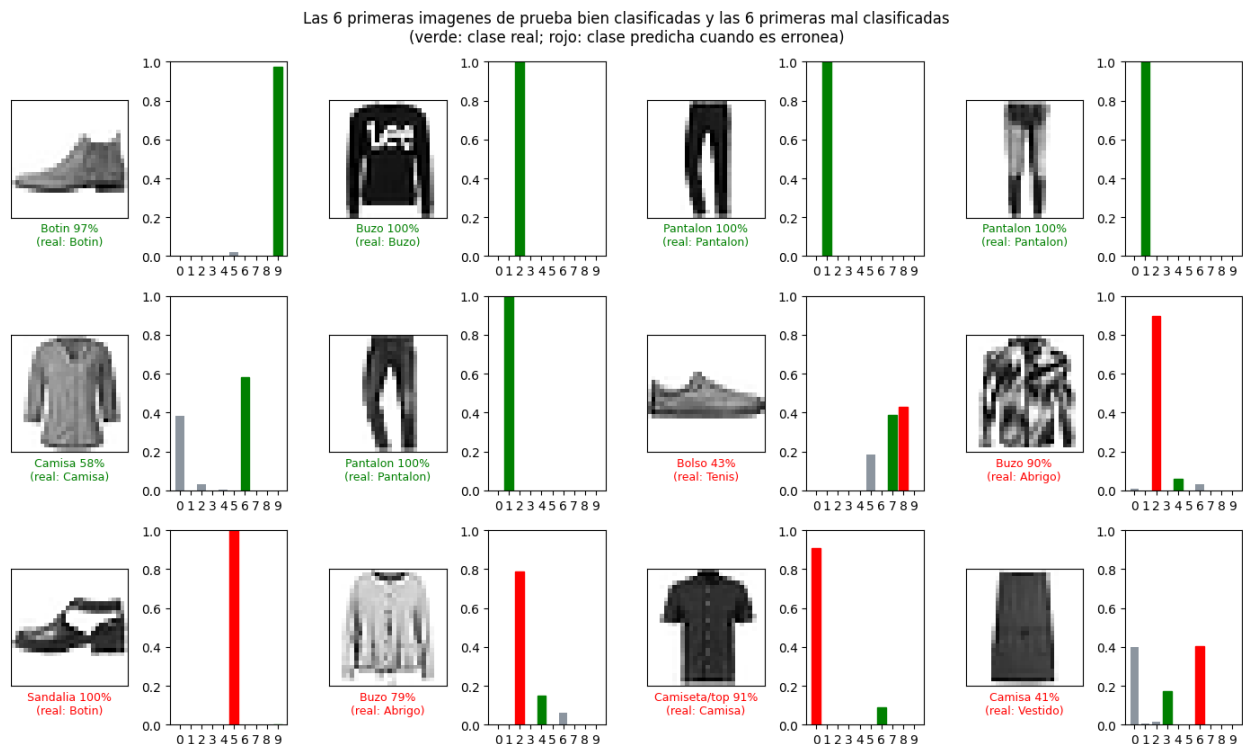


Figura 9. Las seis primeras imágenes de prueba bien clasificadas y las seis primeras mal clasificadas, con las diez probabilidades de salida.

Cinco de los seis aciertos tienen probabilidades de 97 % a 100 %. En los errores hay dos situaciones: en algunos la red duda (un tenis con 43 % de bolso y 38 % de tenis), y en otros se

equivoca con seguridad (un botín clasificado como sandalia con probabilidad cercana al 100 %, una camisa como camiseta con 91 %).

4.4 Conclusiones

1. Con unas quince instrucciones de Keras se construye, entrena y evalúa un clasificador de imágenes: la red del capítulo acierta el 87.4 % de las prendas de prueba.
2. **Cada función corresponde a una ecuación del capítulo, y se comprobó a mano:** Flatten es un reshape, Dense es $g(W^T X + B)$, la pérdida es $-\ln p(y)$, un paso de entrenamiento es $W \leftarrow W - \eta \partial E / \partial W$ y predict es la propagación de la sección 5.4.
3. **Lo que TensorFlow automatiza es la retropropagación.** GradientTape obtiene los gradientes de todos los pesos con la regla de la cadena, sin derivar las ecuaciones de la sección 5.5 para cada arquitectura.
4. **La división en tres conjuntos es necesaria:** la validación detectó el sobreentrenamiento desde la época 8, EarlyStopping lo evitó, y la exactitud de validación (87.9 %) anticipó la de prueba (87.4 %).
5. **La red se equivoca sobre todo con prendas parecidas** y a veces con mucha seguridad: una probabilidad alta no garantiza una respuesta correcta.
6. **Comparado con el Ejercicio 2**, este modelo de una capa oculta de 128 neuronas, sin *Dropout*, obtiene 87.4 % frente a 88.7 % de la red de dos capas ocultas regularizada.

5. Ejercicio 4. Modelo con un dataset de Kaggle y ejemplos de internet

Enunciado: de la librería de Kaggle para Python, baje un dataset, estudie sus características (features) y su rótulo. Genere un modelo y tome ejemplos de internet y analice los resultados.

5.1 Elección del conjunto de datos

Se usó Pokemon with stats (abcsds/pokemon), descargado con kagglehub, la librería oficial de Kaggle para Python. Describe los 800 Pokémon de las seis primeras generaciones (1996 a 2013), incluidas sus formas alternativas, con seis estadísticas de combate numéricas y un rótulo binario, Legendary. La pregunta del modelo es: dadas las seis estadísticas de un Pokémon, ¿es legendario? Se escogió porque tiene características y rótulo claros, porque entre 2016 y 2022 se publicaron tres generaciones más (7, 8 y 9) cuyos datos oficiales están en internet y permiten probar el modelo con ejemplos genuinamente nuevos, y porque plantea problemas reales de aprendizaje de máquina: clases muy desbalanceadas (8 % de legendarios), filas que son variantes de otras y un rótulo que es una decisión narrativa de los creadores del juego.

5.2 Características y rótulo

Tabla 11. Columnas del conjunto y decisión sobre cada una.

Columna	Tipo	¿Se usa?
#, Name	identificadores	No
Type 1, Type 2	categorías (18 tipos)	Se evaluaron; empeoran el modelo
Total	suma de las seis estadísticas	No: es redundante
HP, Attack, Defense, Sp. Atk, Sp. Def, Speed	enteras	Sí: vector X, n = 6
Generation	entera, 1 a 6	No: los ejemplos nuevos son de las generaciones 7 a 9
Legendary	binaria	Rótulo Y

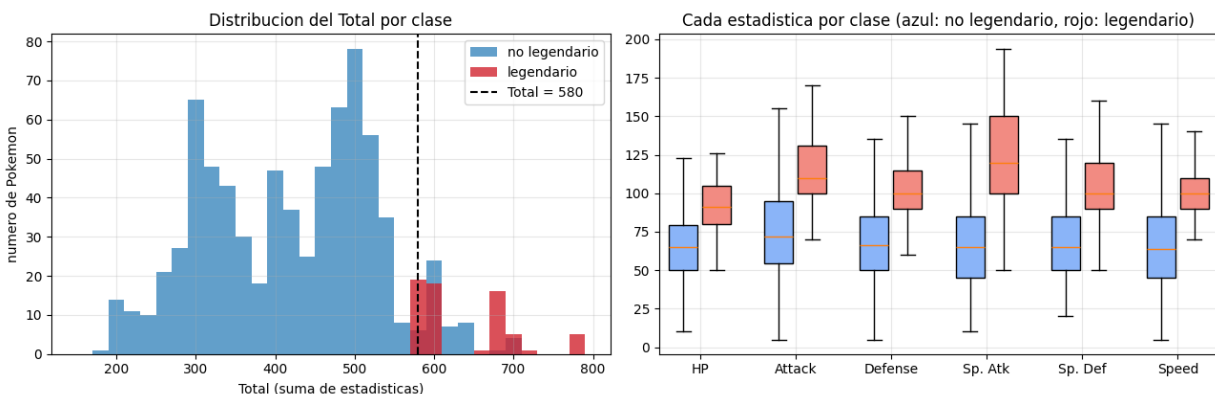


Figura 10. Distribución del Total por clase y de cada estadística.

Todos los legendarios del conjunto tienen un Total de al menos 580, lo que sugiere un clasificador trivial, «legendario si $\text{Total} \geq 580$ », que sirvió como línea base: una red neuronal solo se justifica si mejora a una regla que cualquiera escribiría mirando el histograma.

5.3 Calidad de los datos

Formas alternativas. El conjunto incluye 50 filas de megaevoluciones y regresiones primigenias, transformaciones temporales de un Pokémon existente con estadísticas infladas. De los 48 no legendarios con $\text{Total} \geq 580$, 31 son megaevoluciones de Pokémon comunes (Mega Garchomp, Mega Tyranitar...): ejemplos contradictorios con el patrón que el modelo debe aprender. Se retiraron.

Rótulos. Al revisar los errores de un primer modelo aparecieron Celebi y Keldeo, clasificados como legendarios por la red pero rotulados como comunes. Ambos son míticos, una categoría que el conjunto trata de forma inconsistente (Jirachi o Arceus sí aparecen como legendarios). Contrastando con las listas oficiales de Bulbapedia se encontraron 10 filas mal rotuladas, siempre en la misma dirección: Pokémon legendarios o míticos marcados como comunes. Se adoptó como rótulo «legendario o mítico».

5.4 Modelo: red con dos capas ocultas

Se usó una red 6-16-8-1: dos capas ocultas con ReLU, de 16 y 8 neuronas, y una neurona de salida con sigmoide cuya salida es la probabilidad de ser legendario. Tiene $6 \cdot 16 + 16 = 112$, $16 \cdot 8 + 8 = 136$ y $8 + 1 = 9$ parámetros por capa, 257 en total, acorde con las 524 filas de entrenamiento. Se entrenó con Adam, entropía cruzada binaria y pesos de clase (class_weight) que hacen que un error sobre un legendario pese unas 10 veces más, para compensar el desbalance. Las entradas se estandarizaron con StandardScaler, ajustado solo con los datos de entrenamiento.

Las decisiones de preparación se tomaron con validación cruzada estratificada de 5 particiones, porque con unos 60 legendarios un único conjunto de prueba tendría solo 10 y cada caso movería las métricas varios puntos.

Tabla 12. Validación cruzada de las variantes de preparación de datos (promedio de 5 particiones).

Variante	Modelo	Precisión	Sensibilidad	F1	AUC
A. Datos originales	red	0.588	0.738	0.646	0.963
	Total ≥ 580	0.582	1.000	0.734	—
B. Sin formas alternativas	red	0.774	0.858	0.811	0.979
	Total ≥ 580	0.771	1.000	0.870	—
C. B con rótulo corregido	red	0.792	0.821	0.799	0.976
	Total ≥ 580	0.897	0.985	0.937	—
D. C + 18 variables de tipo	red	0.738	0.729	0.728	0.955
	Total ≥ 580	0.897	0.985	0.937	—

Quitar las formas alternativas sube el F1 de la red de 0.65 a 0.81, más de lo que aportó cualquier cambio de arquitectura; agregar los tipos lo empeora por sobreentrenamiento. Y hay un resultado que conviene decir sin rodeos: con los datos de Kaggle, la regla $Total \geq 580$ supera a la red (F1 de 0.94 frente a 0.80). En las seis primeras generaciones el Total separa las clases casi a la perfección; la red ordena bien a los Pokémon (AUC de 0.98) pero produce más falsas alarmas.

El modelo final se entrenó con la variante C y una división estratificada 70/15/15, con parada temprana sobre la validación. En prueba encontró a los 10 legendarios (sensibilidad de 1.0) pero marcó además a 6 Pokémon comunes (precisión de 0.63, F1 de 0.77, AUC de 0.998); la regla se equivocó una sola vez (F1 de 0.95). Ajustar el umbral de decisión con la validación no cambió el resultado.

5.5 Ejemplos tomados de internet

Se tomaron de PokémonDB, y se contrastaron con Bulbapedia, las estadísticas oficiales de 23 Pokémon de las generaciones 7, 8 y 9, ninguno presente en el conjunto de Kaggle, escogidos para cubrir las categorías que ponen a prueba el modelo: legendarios de portada, legendarios menores y míticos, legendarios en etapa inicial, pseudolegendarios (no legendarios con Total de 600), ultraentes y paradojas, y Pokémon comunes.

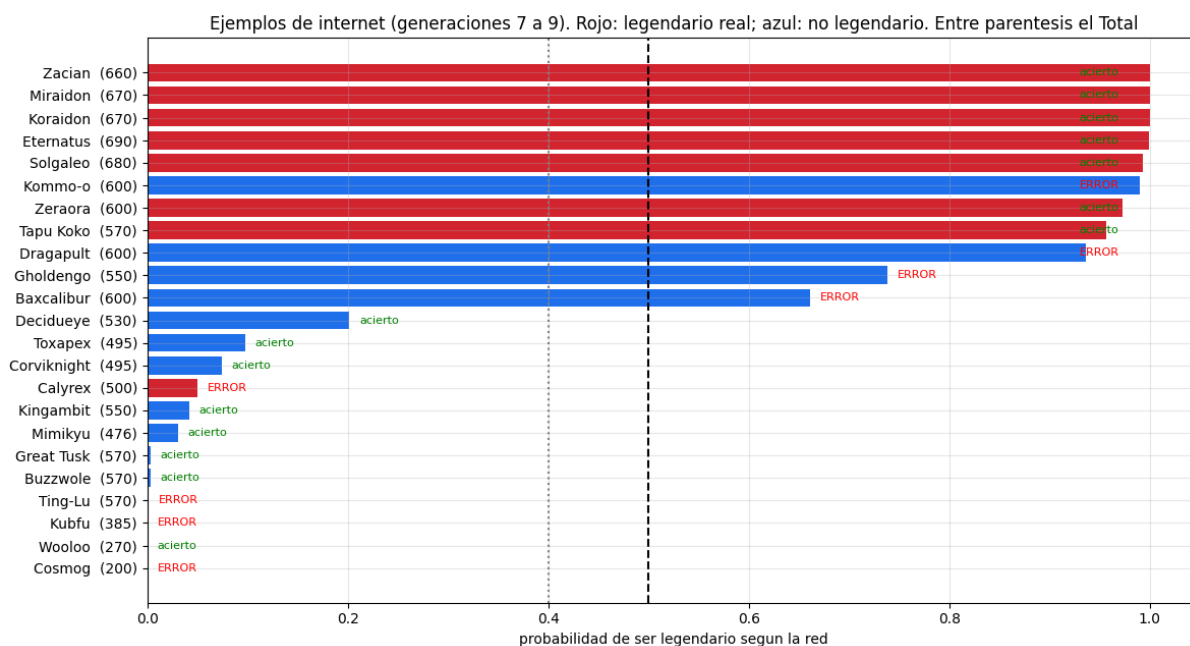


Figura 11. Probabilidad de ser legendario asignada por la red a los 23 ejemplos de internet (rojo: legendario real).

Tabla 13. Aciertos por categoría en los ejemplos de internet.

Categoría	n	Total medio	Aciertos red	Aciertos regla
Legendario de portada	5	674	5	5
Legendario menor	3	547	1	0
Mítico	1	600	1	1

Categoría	n	Total medio	Aciertos red	Aciertos regla
Legendario en etapa inicial	2	293	0	0
Pseudolegendario	3	600	0	0
Ultraente y Paradoja	2	570	2	2
Común	7	481	6	7
Total	23		15	15

5.6 Análisis de los resultados

- **Legendarios de portada y el mítico Zeraora:** los aciertan la red y la regla; tienen estadísticas muy altas, como los legendarios de las primeras generaciones.
- **Legendarios en etapa inicial (Cosmog, Kubfu) y Calyrex:** ningún modelo los detecta. Cosmog tiene un Total de 200 y es numéricamente indistinguible de un Pokémon común; su condición de legendario viene de la historia del juego, y esa información no está en las características. Es el límite conceptual del ejercicio.
- **Pseudolegendarios (Dragapult, Baxcalibur, Kommo-o):** con un Total de 600 y estas seis variables son indistinguibles de un legendario, y ambos modelos los confunden.
- **Qué aprendió la red.** Entre los Pokémon del conjunto con $\text{Total} \geq 550$, los no legendarios tienen mucho más *Attack* (mediana de 132 frente a 101.5) y menos *Sp. Atk* (105 frente a 117.5) que los legendarios. La red aprendió esa asociación: si el *Sp. Atk* de Buzzwole subiera de 53 a 110, su probabilidad de ser legendario pasaría de 0.003 a 0.59. Eso explica sus aciertos (rechaza a Buzzwole, Great Tusk y Kingambit, atacantes físicos) y sus errores (falla con Ting-Lu, legendario atacante físico, y con Gholdengo, común con *Sp. Atk* de 133).

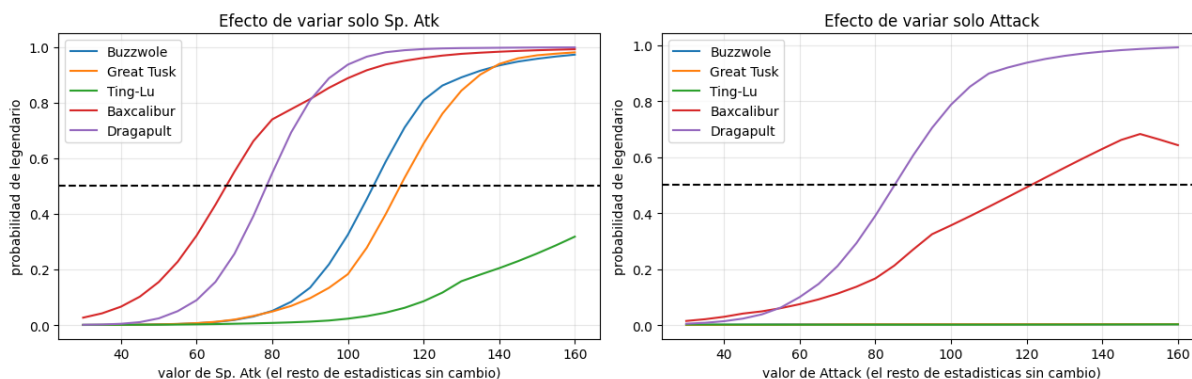


Figura 12. Probabilidad de ser legendario al variar solo el *Sp. Atk* (izquierda) o solo el *Attack* (derecha).

5.7 Conclusiones

1. **Entender los datos aportó más que ajustar el modelo:** retirar 50 filas de formas alternativas subió el F1 de la red de 0.65 a 0.81, y la revisión de los errores reveló 10 rótulos equivocados.

2. **Una red neuronal no es automáticamente mejor que una regla simple:** con los datos de Kaggle la regla $\text{Total} \geq 580$ supera a la red de dos capas ocultas en F1 (0.94 frente a 0.80). Compararse con una línea base es indispensable.
3. **Los ejemplos de internet mostraron lo que las métricas no mostraban:** la red y la regla empatan en aciertos (15 de 23) pero fallan en casos distintos, y el análisis de sensibilidad reveló una correlación de las primeras generaciones que no siempre se cumple en las nuevas.
4. **Un modelo no puede superar la información de sus características:** Cosmog, Kubfu y Calyrex son legendarios por decisión narrativa y ningún modelo entrenado con estas seis variables puede reconocerlos.

6. Ejercicio 5 (punto libre). De la convolución a la red convolucional

Enunciado: punto libre.

6.1 Planteamiento

La sección 5.8 del capítulo introduce la convolución y el max pooling, las dos operaciones en que se basa la visión por computador moderna, pero ninguno de los ejercicios anteriores las usa. Además, los ejercicios 2 y 3 terminaron con una limitación identificada: una red densa, aun con dos capas ocultas, trata cada píxel como una característica independiente y confunde prendas de silueta parecida. El punto libre se dedicó a cerrar ese vacío: implementar desde cero la convolución y el pooling y verificarlas con los ejemplos del capítulo, reproducir la figura 5.24, entrenar una red convolucional (CNN) y compararla con la red densa de dos capas ocultas del Ejercicio 2, y examinar por dentro la red entrenada.

6.2 Convolución y pooling desde cero

La convolución se programó de dos formas, con ciclos, siguiendo literalmente el capítulo, y vectorizada. Ambas reproducen exactamente la matriz 4×4 que resulta de aplicar el filtro de bordes verticales a la imagen de 6×6 del capítulo, y coinciden con `correlate2d` de SciPy y `tf.nn.conv2d`. Se aclaró un detalle de nomenclatura: lo que el capítulo y las librerías de redes llaman convolución es, en rigor, una correlación cruzada, porque no invierte el filtro. El max pooling reproduce la matriz 2×2 de la figura 5.23 y coincide con `MaxPooling2D` de Keras.

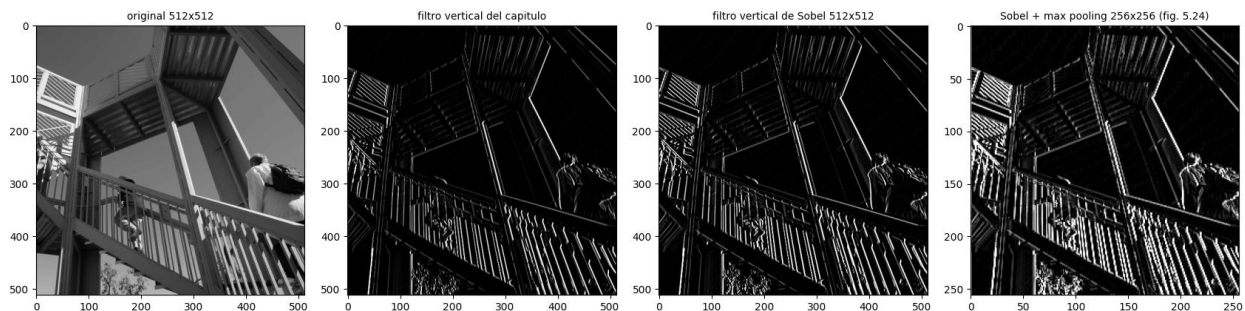


Figura 13. Reproducción de la figura 5.24: imagen original, filtro vertical del capítulo, filtro de Sobel y max pooling a 256×256 .

La figura 5.24 se reprodujo con la imagen *ascent* de SciPy, que es la que usa el libro de Moroney: el filtro vertical resalta las líneas verticales de la escalera y el max pooling reduce la imagen de 512×512 a 256×256 conservando esos rasgos.

6.3 Red convolucional

La CNN tiene dos bloques de convolución y pooling (32 y 64 filtros de 3×3), seguidos de una capa densa de 128 neuronas con *Dropout* y la salida softmax: 225 034 parámetros, un número similar a los 235 146 de la red densa de dos capas ocultas. La primera capa convolucional tiene solo 320 parámetros frente a los 200 960 de la primera capa oculta de la red densa, gracias al **peso**

compartido: el mismo filtro se aplica en todas las posiciones de la imagen. Las dos redes se entrenaron en las mismas condiciones (Adam, lotes de 64, parada temprana con paciencia de 3 épocas).

Tabla 14. Red densa de dos capas ocultas frente a la red convolucional, en el conjunto de prueba.

Modelo	Parámetros	Épocas	Tiempo en CPU	Pérdida	Exactitud
Densa 784-256-128-10 (Ejercicio 2)	235 146	12	48 s	0.338	87.97 %
CNN	225 034	13	208 s	0.265	91.06 %

La CNN reduce el error de prueba de 12.0 % a 8.9 %, un 26 % menos errores. La exactitud de la red densa aquí (88.0 %) es algo menor que en el Ejercicio 2 (88.7 %) porque se entrenó con los criterios de esta comparación (lotes de 64 y paciencia de 3), y queda dentro del rango observado entre semillas en ese ejercicio.

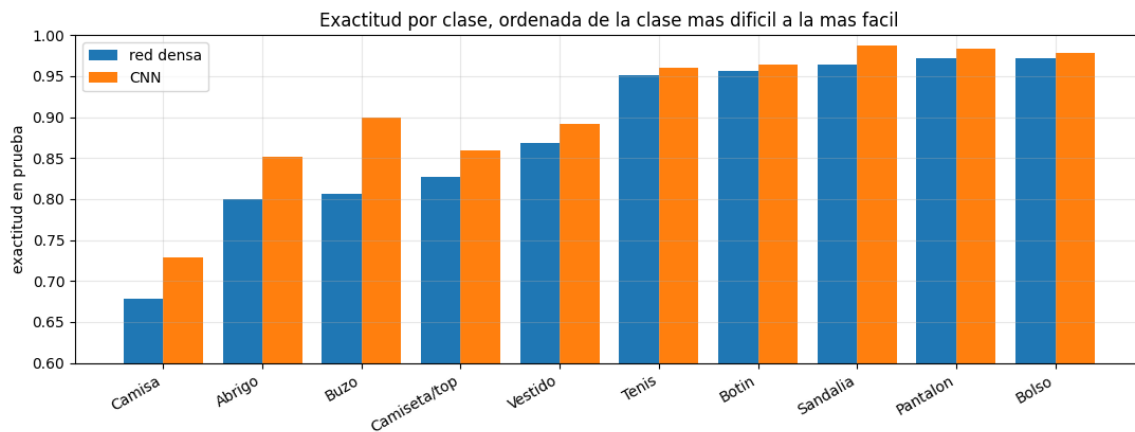


Figura 14. Exactitud por clase de la red densa y de la CNN.

La CNN mejora en las diez clases, sobre todo en las prendas superiores que la red densa confundía: buzo (+9.3 puntos), abrigo (+5.2), camisa (+5.0) y camiseta (+3.2), que se distinguen por detalles locales como la manga, el cuello o los botones. La camisa sigue siendo la clase más difícil (72.9 %), lo que indica un límite de los datos más que del modelo.

6.4 Dentro de la red entrenada

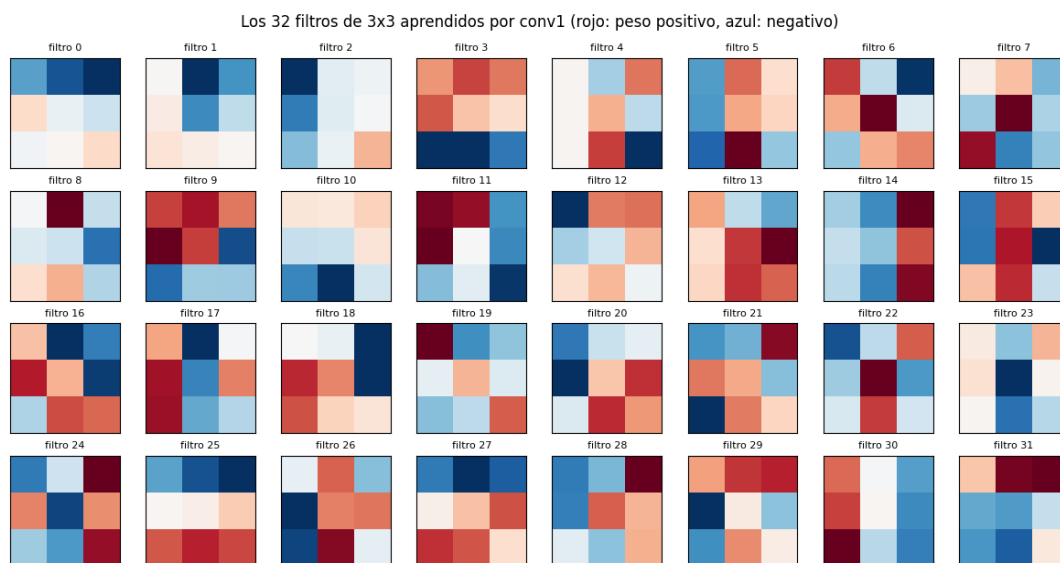


Figura 15. Los 32 filtros de 3×3 aprendidos por la primera capa convolucional.

Los 32 filtros partieron de valores aleatorios y varios terminaron con la estructura de detectores de bordes: uno tiene correlación de $+0.96$ con el filtro de bordes verticales del capítulo y otro de -0.97 con el de bordes horizontales. Es lo que ilustra la figura 5.21: las primeras capas aprenden rasgos simples. Finalmente se aplicaron a mano, con la función de convolución programada al comienzo, los 32 filtros aprendidos a una imagen, más el sesgo y la ReLU: el resultado coincide con la salida de la capa Conv2D con una diferencia de 8×10^{-8} .

6.5 Conclusiones

1. Las implementaciones propias reproducen todos los ejemplos numéricos de la sección 5.8 y la figura 5.24.
2. La CNN supera a la red densa de dos capas ocultas: 91.1 % frente a 88.0 % con un número de parámetros similar, una cuarta parte menos de errores, gracias a que aprovecha la estructura espacial de la imagen.
3. Los filtros aprendidos se parecen a los diseñados, y una capa convolucional es exactamente la convolución del capítulo seguida de un sesgo y una función de activación.

7. Uso de herramientas de IA Generativa

Los parámetros de entrega del curso establecen que, cuando se utilicen herramientas de IA Generativa, se deben hacer comentarios sobre qué herramientas se emplearon, los prompts utilizados y hasta qué punto sirvieron para el desarrollo de la documentación, los ejercicios o los proyectos. Este capítulo atiende esa exigencia.

7.1 Herramientas empleadas

Tabla 15. Herramientas empleadas en el desarrollo del capítulo.

Herramienta	Uso que se le dio	Utilidad observada
Asistente de IA Chatgpt	Verificaciones y explicaciones de tecnología.	Siempre que sus afirmaciones se contrastaran con los resultados (sección 8.3).
Documentación oficial de TensorFlow/Keras, scikit-learn, SciPy y kagglehub	Consulta de la interfaz de las funciones utilizadas.	Necesaria para confirmar el comportamiento de funciones como EarlyStopping, class_weight, GradientTape y load_model.
Bates y Watts (1988) y el conjunto Puromycin de R	Fuente de la tabla del experimento del Ejercicio 1 y de los parámetros de referencia de Michaelis-Menten.	Permitió verificar el ajuste del modelo físico.
PokémonDB y Bulbapedia	Estadísticas oficiales de los 23 ejemplos de internet y listas oficiales de legendarios y míticos.	Indispensable para el Ejercicio 4.

7.2 Prompts utilizados

Las solicitudes principales hechas a la herramienta fueron del siguiente tipo:

- Proponer una tabla de datos de un experimento real y construir con ella un modelo de red neuronal de dos capas ocultas que permita hacer predicciones.
- Explicar cada una de las funciones e instrucciones de TensorFlow usadas en la clasificación de Fashion MNIST.
- Verificar los resultados obtenidos y explicar las diferencias encontradas.

7.3 Alcance y limitaciones observadas

Durante este trabajo ocurrió varias veces, y los textos se corrigieron al contrastarlos con las salidas de los programas:

- Describió la salida de la red sin entrenar del Ejercicio 3 como un reparto casi uniforme de 0.1 por clase, pero la salida real asignaba 0.42 a una sola clase.
- Afirmó que la predicción del Ejercicio 1 variaba menos de 2 cuentas/min² entre semillas, cuando el rango real era de 5.2 (la desviación estándar sí es de 1.6).

- En el Ejercicio 3 escogió imágenes de ejemplo que resultaron ser en su mayoría errores, lo que daba una impresión equivocada del modelo; se reemplazaron por seis aciertos y seis errores identificados explícitamente.

8. Programas, repositorio y ejecución

8.1 Estructura de los archivos

```
5. Redes neuronales/  
├── README.md  
├── requirements.txt  
├── Documento_Neural_Neuronales.pdf  
├── notebooks/  
│   ├── Ejercicio_1_Experimento_Red_Dos_Capas_Ocultas.ipynb  
│   ├── Ejercicio_2_Fashion_MNIST_Dos_Capas_Ocultas.ipynb  
│   ├── Ejercicio_3_Fashion_MNIST_Funciones_Instrucciones.ipynb  
│   ├── Ejercicio_4_Kaggle_Pokemon_Legendarios.ipynb  
│   └── Ejercicio_5_Convolucion_Pooling_CNN.ipynb  
└── img/          figuras generadas por los notebooks
```

8.2 Ejecución en Google COLAB

Cada notebook se puede abrir directamente en Colab con los siguientes enlaces; para los ejercicios 2, 3 y 5 se recomienda activar la GPU (*Entorno de ejecución* → *Cambiar tipo de entorno*).

- [Ejercicio 1. Experimento con una red de dos capas ocultas](#)
- [Ejercicio 2. Fashion MNIST con dos capas ocultas](#)
- [Ejercicio 3. Fashion MNIST: funciones e instrucciones](#)
- [Ejercicio 4. Dataset de Kaggle](#)
- [Ejercicio 5. Convolución, pooling y CNN](#)

8.3 Ejecución local con Jupyter

```
pip install -r requirements.txt  
jupyter notebook notebooks/
```

Tiempos aproximados en CPU: Ejercicio 1, 2 minutos; Ejercicio 2, 20 minutos (la mayor parte es la comparación de arquitecturas con tres semillas); Ejercicio 3, 4 minutos; Ejercicio 4, 5 minutos; Ejercicio 5, 5 minutos. Todos los notebooks fijan semillas aleatorias, de modo que en CPU los resultados se repiten; en una GPU las cifras de los ejercicios con TensorFlow pueden variar en décimas de punto porcentual.

9. Bibliografía

- [1] Martínez, J. J. (2026). *Introducción a las redes neuronales artificiales*, capítulo 5. Material del curso Inteligencia Artificial y Minirobots, Universidad Nacional de Colombia.
- [2] Moroney, L. (2020). *AI and Machine Learning for Coders: A Programmer's Guide to Artificial Intelligence*. O'Reilly.
- [3] Ng, A. *Machine Learning*. Coursera.
- [4] He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of CVPR*.
- [5] Treloar, M. A. (1974). *Effects of Puromycin on Galactosyltransferase in Golgi Membranes*. Tesis de maestría, Universidad de Toronto.
- [6] Bates, D. M. y Watts, D. G. (1988). *Nonlinear Regression Analysis and Its Applications*. Wiley.
- [7] Rumelhart, D., Hinton, G. y Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533-536.
- [8] Xiao, H., Rasul, K. y Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747.
- [9] Barradas, A. *Pokemon with stats* [conjunto de datos]. Kaggle, identificador abcsds/pokemon.
- [10] PokémonDB, <https://pokemondb.net/pokedex>, y Bulbapedia, <https://bulbapedia.bulbagarden.net>. Consultados en septiembre de 2026.